

Unit 5 – Equivalence Partitioning: Investigation and Security Analysis

1. Overview of the Activity

This activity required executing and analysing the provided `equivalence.py` script in a Jupyter Notebook environment. The script implements a general equivalence partitioning algorithm that groups elements of an iterable into equivalence classes according to a user-defined relation.

The supplied relation in `test_equivalence_partition()` is:

```
lambda x, y: (x - y) % 4 == 0
```

This defines equivalence as congruence modulo 4: two integers are equivalent if their difference is divisible by 4.

Equivalence Partitioning (EP) is a specification-based test design technique formally recognised in ISO/IEC/IEEE 29119-4:2021 as a method for reducing the number of test cases while maintaining behavioural coverage (ISO/IEC/IEEE, 2021). The technique assumes that inputs within the same partition will exhibit equivalent behaviour under test.

2. Observed Output and Mathematical Verification

Running:

```
range(-3, 5)
```

produces four equivalence classes:

- $\{0, 4\}$
- $\{1, -3\}$
- $\{2, -2\}$
- $\{3, -1\}$

These correspond to the four residue classes modulo 4:

- $r = 0$
- $r = 1$
- $r = 2$
- $r = 3$

Verification confirms correctness:

- Within-class example:
 $(1 - (-3)) \% 4 = 4 \% 4 = 0 \rightarrow \text{True}$
- Across-class example:
 $(0 - 1) \% 4 = -1 \% 4 = 3 \rightarrow \text{False}$

The relation is:

- Reflexive ($x \sim x$)
- Symmetric (if $x \sim y$ then $y \sim x$)
- Transitive (if $x \sim y$ and $y \sim z$ then $x \sim z$)

Therefore, it satisfies the formal requirements of an equivalence relation, justifying disjoint partitioning.

3. Further Investigations

3.1 Changing the Modulus

Replacing `% 4` with `% 3` over `range(-5, 6)` produces three classes:

- $\{-3, 0, 3\}$
- $\{-5, -2, 1, 4\}$
- $\{-4, -1, 2, 5\}$

This confirms that the number of partitions is determined directly by the equivalence relation specification, not by implementation details.

3.2 Non-Numeric Domain

Applying:

```
relation = lambda s1, s2: len(s1) == len(s2)
```

to:

```
["cat", "dog", "rat", "elephant", "ant", "bat", "lion", "tiger"]
```

produces partitions by string length:

- Length 3: {"ant", "bat", "cat", "dog", "rat"}
- Length 4: {"lion"}
- Length 5: {"tiger"}
- Length 8: {"elephant"}

This demonstrates that equivalence partitioning is domain-agnostic and can be applied to structural properties such as input length — highly relevant in secure validation contexts.

3.3 Testing a Non-Equivalence Relation

Defining:

relation = lambda x, y: abs(x - y) <= 1

violates transitivity. The resulting partitions become unstable, illustrating a critical constraint:

Equivalence Partitioning is valid only when the relation is a true equivalence relation. Mis-specification can generate misleading coverage and false confidence.

4. Security Relevance in Secure Software Development

Equivalence Partitioning is a foundational black-box technique within specification-based test design (ISO/IEC/IEEE, 2021). In security contexts, it supports:

- Structured coverage of valid and invalid input domains
- Systematic testing of rejection paths
- Reduced combinatorial explosion in input validation scenarios

For example:

- Password lengths (valid vs too short/too long)
- Input format classes (valid email vs malformed variants)
- Numeric range checks (acceptable vs overflow-triggering values)

OWASP's Web Security Testing Guide emphasises structured input validation testing as a control against injection and malformed input exploitation (OWASP, 2020). Partitioning supports this by ensuring representative coverage across behaviourally distinct input groups.

However, Equivalence Partitioning has limitations:

- It does not inherently test boundary values (which are often where security defects cluster).
- Poorly defined partitions may obscure edge-case vulnerabilities.
- It may provide false assurance if representatives are not carefully selected.

Therefore, EP must be combined with Boundary Value Analysis (BVA) to identify off-by-one errors and limit-condition flaws — common precursors to buffer overflows or validation bypasses.

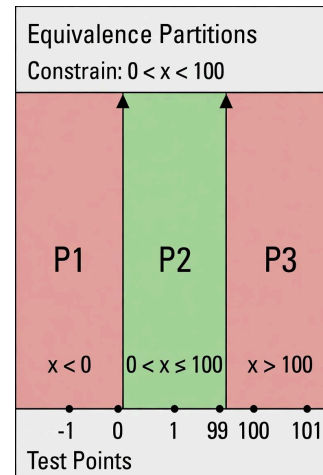
5. Relationship to Complexity and Attack Surface

Unpartitioned input domains increase the number of possible execution paths. When combined with high cyclomatic complexity (McCabe, 1976), the risk of untested logic branches increases.

Equivalence Partitioning reduces uncertainty by grouping behaviourally similar inputs. However, if complexity remains high, even representative testing may miss rare execution paths.

Thus:

- Partitioning reduces input-domain uncertainty.
- Complexity reduction reduces control-flow uncertainty.



Secure testing requires managing both.

6. Team Discussion and Reflection

During team discussion, we considered whether Equivalence Partitioning alone provides sufficient assurance in security-sensitive systems. We concluded that while EP drastically reduces test volume, adversaries frequently target boundary conditions rather than nominal class members.

We also debated whether automated partition-based testing might encourage complacency. The consensus was that EP is most effective when integrated into a broader layered testing strategy that includes:

- Boundary Value Analysis
- Static analysis (e.g., linting and security scanning)
- Dynamic adversarial testing

The key insight from discussion was that partitioning is a risk-management tool, not a completeness guarantee.

7. Critical Reflection

This activity demonstrates that secure testing is not merely about automation or coverage metrics. It is about carefully defining behavioural equivalence and validating that definitions reflect real security boundaries.

Equivalence Partitioning formalises assumptions about input behaviour. When those assumptions are correct, testing efficiency improves without sacrificing meaningful coverage. When they are incorrect, entire vulnerability classes may remain invisible.

Secure software development therefore requires:

- Formal reasoning about equivalence
- Explicit threat modelling of input domains
- Complementary boundary and adversarial testing

Testing reduces uncertainty — but only when the partitions themselves are correctly specified.

References

ISO/IEC/IEEE (2021) *ISO/IEC/IEEE 29119-4:2021 Software and systems engineering — Software testing — Part 4: Test techniques*. Available at:

<https://www.iso.org/standard/79430.html> (Accessed: 22 February 2026).

McCabe, T. J. (1976) 'A Complexity Measure', *IEEE Transactions on Software Engineering*, 2(4), pp. 308–320.

OWASP (2020) *OWASP Web Security Testing Guide v4.2*. Available at:

<https://owasp.org/www-project-web-security-testing-guide/> (Accessed: 22 February 2026).

University of Essex Online (2026) *Specification-based Test Design Techniques – Transcript*. Secure Software Development (SSDCS_PCOM7E).